DEVOPS & CLOUD [SWE] · SCALER ACADEMY

$ cat notes.md

DevOps & Cloud [SWE] — Class 7 Notes

LECTURE TITLE	**Docker Images & Dockerfile (Deep Dive)**
DATE/TIME	**25 August 2026, 11:00 AM**
DURATION	**120 minutes**
INSTRUCTOR	**Nensi Ravaliya**
BIG IDEA	**Break things on purpose. Every error message in this class is an interview answer.**
SOURCES	**this class's transcript + your screenshots in `Class-06/` + the instructor's repo Nency-Ravaliya/devops-heros → `session6-7-docker/`**

WHAT'S INSIDE — 18 SECTIONS

00 ⚡ The 60-Second Version

01 🗒️ Housekeeping

02 🎮 Recap Quiz — Everything From Class 6

03 🆕 Docker Hosts Can Talk To Each Other

04 🐋 Docker Images — Immutability & the Blueprint

05 ♾️ Container Lifecycle — The Full Hands-On

06 🔷 The Mess That Force-Removing Leaves Behind

07 🧹 The Prune Commands — Reclaiming Your Disk

08 🐋 docker image inspect — And Why It Matters At Work

09 🐋 docker rmi — Removing Images, and All The Ways It Fails

10 🐋 docker exec — Getting Inside a Running Container

11 🔷 Class Engagement (The Comedy Section)

12 🔁 Dockerfile Instructions — Recap With New Details

13 🏁 Multi-Stage Dockerfiles — Today's Main Topic

14 📝 Homework

15 🔁 Her Own Recap

16 📒 Cheat-Sheet

17 🗒️ Notes on the Transcript Itself

Rebuilt from the raw class transcript — nothing cut, just organised, de-noised and made readable. `theme: Neon Nights`

00 ⚡ The 60-Second Version

1. **Your Docker host isn't alone.** It can connect to *other* Docker hosts — which is exactly what Kubernetes does inside a cluster.
2. **The container lifecycle has more states than you think:** created → running → paused → stopped/exited → removed. And `created` ≠ `running`.
3. `docker ps` **hides stopped containers.** `docker ps -a` **shows everything.** Half of all Docker confusion lives in that one flag.
4. `start` **vs** `restart`**:** `start` boots from a fresh PID 1; `restart` resumes from the state you stopped at. Both work after a stop.
5. **You can't** `rm` **a running container** — stop it first, or force it with `-f`.
6. **Forcing things leaves rubbish behind:** dangling images, unused containers, zombie processes. `docker system prune` is the broom.
7. `docker exec -it <id> /bin/bash` **drops you inside a container** — and it's a tiny Linux box in there.
8. **Multi-stage builds exist because** `node_modules` **is enormous.** Build in stage 1, copy only the output into stage 2. Two keywords do all the work: `AS builder` and `--from=builder`.

◆ ◆ ◆

01 🗒️ Housekeeping

The seating shuffle

She reorganised the room before starting:

> 😄 **CLASS MOMENT**
>
> *"Team A people, can you come forward this line? Come here, come here, Team C — I just want to see how many people… I need to see each and every one. There are still people I cannot see, but that's okay."*
>
> And: *"Can anyone help me on this projector? …Yeah, I know. I did not share it yet."* 😄

📌 A transcript upgrade

From this class onward the transcript header shows a real pipeline:

```
Generated by:  nase.24bcs10075@sst.scaler.com
Model use:     groq, whisper-large-v3-turbo
```

💡 GOOD TO KNOW

Classes 1–6 were transcribed with a noisier tool (hence all the stray Ukrainian and Telugu). From Class 7 it's **Whisper large-v3-turbo**, and the quality jump is obvious — barely any garbled fragments left.

😄 The 11 AM problem

😄 CLASS MOMENT

"Everyone wants to call me after 11, because they know I'm in class 9 to 11. But we scheduled it from 11 to 1. So, sorry for that."

◆ ◆ ◆

02 🎮 Recap Quiz — Everything From Class 6

She ran a rapid-fire round with Teams A / B / C before starting anything new.

"What is Docker? Why do we need it?"

🎯 INTERVIEW ALERT

She pushed past the obvious: *"We have servers, we can store the code, we have Git and GitHub — why do we need Docker?"*

Then: *"What's the first word that comes into your mind when you hear Docker?"* → **containerization** · **images** · **Kubernetes**.

✅ LOCK THIS IN

Her full definition, worth learning as a sentence:

"Docker is a platform which ships your application code, dependencies, requirements and environments inside one container, so that you can deploy it easily, manage complex deployment easily, and scale your services up and down easily — even if you want to deploy multiple components at once with different environments or different dependencies."

The killer example, again: Python 3 · Node 18 · Node 20 — *"You need to run both versions on the same server? Then you use containers."*

The rapid-fire board

Question	Answer
Command to see all your images?	`docker images`
Command to list containers?	`docker ps`
…and to include stopped/exited ones?	`docker ps -a`
Default tag if you don't give one?	`latest`
What's a Docker registry?	where all the images live — **Docker Hub**
What's a Docker image?	a **blueprint** of the application · a **read-only template**
Where do we pull from?	**Docker Hub**
Which architecture does Docker follow?	**client–server**
Who's the client?	**you** — the user running the commands
What's inside the Docker host?	**images, containers, and the Docker daemon**

😄 The one that stumped everyone: what does the daemon *do*?

😄 **CLASS MOMENT**

"Docker daemon — what is it?" Student: *"It's a detach mode. Background processes in detach mode."*

"That's one of the things we include in Docker daemon, but what does it DO? What's the functionality? Anyone? Anyone, guys?"

✅ **The answer:** *"Docker daemon is the heart of Docker. Everything you perform inside Docker — `docker run`, `docker pull nginx`, `docker stop`, `start`, `restart`, creating a volume, creating a multi-container, creating a Docker Compose, making containers up and down, killing processes, executing commands inside a container — anything you do is handled by the Docker daemon."*

🔍 **CLARIFIED**

Clarified, because the student's confusion is completely understandable: the word "daemon" in Linux means *a background service*, so "background process" isn't wrong — it's just too small an answer. And the `-d` flag is short for **detach**, not daemon (though it *used* to be called daemon mode — see §5). Two different uses of the same word family, in the same tool. Fun.

◆ ◆ ◆

03 🆕 Docker Hosts Can Talk To Each Other

This was genuinely new material, and it's the bridge to Kubernetes.

🎯 **INTERVIEW ALERT**

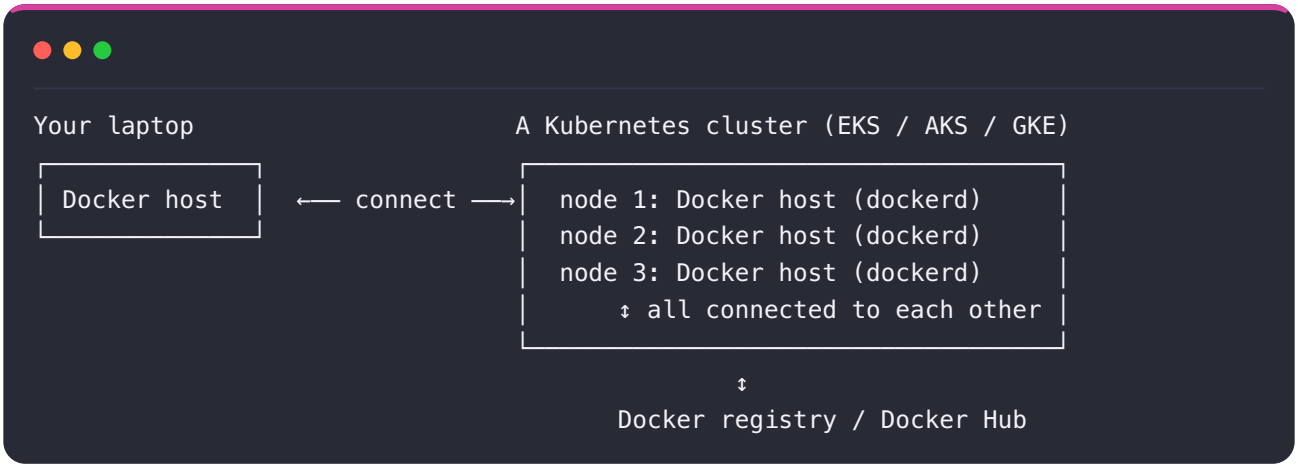
Her setup: *"This is my Docker host. Then I can connect with your laptop, your Docker host. This is possible, by the way."*

⚠️ **WATCH OUT**

The misconception she wanted killed: *"If someone tells you that you can only install a Docker host on your laptop and you're not able to connect with other Docker hosts — then you're wrong. We are able to connect with multiple Docker hosts."*

What you can do once they're connected: share images, save images between hosts.

🧠 Where this actually matters — Kubernetes



✅ **LOCK THIS IN**

Her point: *"In big environments we have Kubernetes, and inside Kubernetes we have a container runtime — `dockerd`. That particular thing we use in Kubernetes is nothing but a Docker host. So inside Kubernetes we also have a Docker host, and it's a cloud-based cluster — Kubernetes on a server, or EKS, AKS, GKE. Those Docker hosts connect with all the Docker hosts inside the cluster."*

💡 **Why you care now:** it means "a Docker host" isn't your laptop specifically. It's a role. Every worker node in a cluster is playing it.

🆕 Registry extras: extensions & plugins

💡 GOOD TO KNOW

Docker Hub holds more than images: **extensions and plugins** too.

Where you'll use them: *"This we use when we're using CI/CD pipelines. Anyone use Jenkins? They know these plugins."*

◆ ◆ ◆

04 🐋 Docker Images — Immutability & the Blueprint

🧠 Immutability, via Java

🎤 SAID IN CLASS

"Docker image is an immutable object. Immutability — do you know it in Java? It will not be changed."

🎯 INTERVIEW ALERT

Her question: why do we say a Docker image is immutable?

```
Dockerfile  →  docker build  →  IMAGE  →  docker run  →  CONTAINER
                                  ↑
                    you now edit your source code...
                    ...and NOTHING changes in the running container
```

✅ **Because the image is a frozen snapshot.** *"You need to rebuild the image. If you rebuild it, then those new changes will reflect inside your container. That's why we call Docker images a read-only template and immutable."*

🧠 Blueprint, via architecture

🧠 PICTURE THIS

Picture this. *"Do you know the blueprint of a building? While architects are designing a building, they create a blueprint. Maybe you've seen it in movies — there's a building, four or five architects with yellow hats, and they go 'okay, this will be the building.' They sketch it, they colour it, and they show it to the client. Like a township — they draw the whole township, colour it, and give it to the client: 'see, this is what we're going to build.'"*

✅ **A Docker image is that drawing.** It fully specifies the thing without being the thing.

🐍 The five manual steps → the five Dockerfile lines

She rebuilt the blueprint argument from scratch, this time with **Python**:

> 🎯 **INTERVIEW ALERT**
>
> *"Imagine you have a server and you don't have Docker. You need to deploy a Python application. What do you do?"*

#	On a bare Linux server, by hand	The Dockerfile line
1	install **Python**	`FROM python:3.11`
2	create the **Python environment**	*(the image is the environment)*
3	install all the **libraries** from `requirements.txt`	`RUN pip install -r requirements.txt`
4	`git clone` your code	`COPY . .`
5	**run** it — `python app.py`	`CMD ["python", "app.py"]`

> ✅ **LOCK THIS IN**
>
> *"The five things we need to run an application — five instructions — we just put in a Dockerfile. That's why we call a Docker image the blueprint of your application."*
>
> ⚠️ And the same caveat as Class 6: *"Same commands, but with Docker instructions"* — `FROM`, `RUN`, `WORKDIR`, `COPY`, `EXPOSE`, `ENTRYPOINT`, `CMD`.

🎯 The "trigger point" per stack

She made the class name the start command for each stack — this is the `CMD` / `ENTRYPOINT` line:

Stack	Trigger point
Python	`python app.py`
nginx	`nginx -g "daemon off;"` — *"it triggers the default `nginx.conf` to run your nginx application"*
Apache	same idea (`httpd-foreground`)
Node.js	`npm install` then `npm run build` / `npm run dev`

◆ ◆ ◆

05 ∞ Container Lifecycle — The Full Hands-On



The `created` state — a container that exists but has never run

She used a command you don't normally reach for, specifically to show the state:

```
docker container create --name mynginx nginx
```

✅ **LOCK THIS IN**

"Container is created." But `docker ps` shows nothing. `docker ps -a` shows it as **Created**.

🎤 SAID IN CLASS

On PID 1: *"Whenever you create a container, that means PID 1 is created but not started yet. When you execute `docker run`, it starts the initialisation process — the PID 1 service — and it goes into the running stage."*

😄 The 5-second container

She ran it and it died almost instantly:

😄 CLASS MOMENT

"It stopped. Started and stopped in 5 seconds. I need to start it and you need to check it in 5 seconds."

⚠️ **Why:** *"I did not add `-d`, `-it`, or even `-p`. If you want your container to keep running, you need to add an entry point inside your Dockerfile. I did not do anything — I just created it. So it will create and it will die."*

✅ **Lesson:** a container without a long-running foreground process (PID 1) exits immediately. That's not a bug — it's the definition.

Then:

```
docker start mynginx     # created → running
docker ps                # ✅ now it shows up (not just ps -a)
```

🧪 The best experiment of the class: does closing the terminal kill it?

```
docker run --name mynginx nginx     # NO -d, so it runs in the FOREGROUND
```

With no `-d`, you see all the container's logs streaming live. Then:

Action	What happens
`Ctrl + C`	❌ **container STOPS**
close the terminal window	✅ **container KEEPS RUNNING** — the class guessed wrong on this one
reboot / shut down the laptop	❌ **container stops**

🧪 TRY IT YOURSELF

She had the class prove it: *"Open two terminals. In one, run `docker run --name mynginx nginx`. Then close that terminal. In the second terminal, run `docker ps -a`. Are you able to see a running container or a stopped container?"* → **running.**

✅ LOCK THIS IN

And the rule that follows: *"If you run your container without detached mode, then if something happens inside your server or laptop, your container will be stopped. But in detached mode — whether you close it or not — until you shut down your laptop it will be up. In real time we're not doing this on laptops, we're doing it on servers, and for that reason we have to use `-d` detach mode. Always. Always."*

💡 GOOD TO KNOW

Vocabulary history: *"Previously when Docker was invented they called it daemon mode. But now, after maybe 10 or 20 years, they call it detach mode."*

`docker logs` — seeing output from a detached container

🎤 SAID IN CLASS

"The people who made it, they are not fools. If you're running the container in detached mode and you still want to see the logs — then this is the command."

```
docker logs <container-id>
```

`--name` and Docker's random-name generator

🎯 INTERVIEW ALERT

Her question: *"If you're not specifying `--name`, which name will it take?"* → **a random one.**

She proved it live:

```
docker run -d nginx
docker ps
# NAMES:  competent_thompson    ← 😄
```

✅ LOCK THIS IN

Why always name your containers: *"If you don't have the ID, you have to type this name — you have to copy-paste it. So usually you need to give a container name whenever you're running a container."*

```
docker run -d --name mynginx1 -p 8080:80 nginx
docker stop mynginx1        # ✅ works — name OR id
```

💡 She mentioned her own naming from a Kubernetes demo: `live-demo-worker`, `worker-node` — descriptive names, not `competent_thompson`.

⏸️ Pause vs 🛑 Stop — a real distinction

	`docker pause`	`docker stop`
Container still "running"?	✅ **yes**	❌ no
Internal services	**frozen** — *"not functioning as required"*	shut down
Data	**state saved** — *"no data will be leaked or vanished"*	*"please save your file, save your state, and then stop"* — a **graceful shutdown**
When you'd use it	*"suddenly you have a problem with resources — less memory, your laptop is lagging"*	you're done with it for now

✅ LOCK THIS IN

Both preserve your data. The difference is whether the container is still *there*, holding its process, or shut down cleanly.

🎯 `start` vs `restart` — she made this a task

🧪 TRY IT YOURSELF

The experiment she assigned:

```
docker run -d --name mynginx nginx
docker stop mynginx
docker restart mynginx     # does this work?
docker stop mynginx
docker start mynginx       # does THIS work?
```

✅ **Answer: both work.** But they do different things:

	`docker start`	`docker restart`
Where it begins	**fresh from PID 1**	resumes **the state you left**
Use it when	you need to *"completely refresh your application"*	you just stopped it and want it back
Which is normal	less common	⭐ *"If you stopped it, we use restart."*

🧠 **PICTURE THIS**

Her analogy: `docker start` is like a full power-cycle — everything from scratch. `docker restart` is picking up where you left off. *"When you're stopping your container, the daemon stores the state, saves your file data and volume — that's the graceful shutdown. Once you restart it, it restarts from the state you left."*

⚠️ You can't remove a running container

```
docker run -d --name nginx1 nginx
docker rm <container-id>          # ❌ ERROR
```

🎯 **INTERVIEW ALERT**

Her framing: *"I'm jumping from a running stage directly to the remove stage. Can you see the error? Either you need to stop it and then delete it. There's only one way, right?"*

✅ **Or force it:**

```
docker rm -f <container-id>       # ✅ no error — stop + remove in one
```

"So I can remove a container forcefully without stopping it — by adding just one flag, `-f`."

😄 The bug she left in on purpose

```
docker run -d --name mynginx1 nginx
# → Error: the name "mynginx1" is already in use
```

😄 CLASS MOMENT

She'd "already removed" the container and was baffled. The class debugged it: *"Change the name."* / *"Already existing container."* → correct.

Then the confession: *"Maybe it was not removed, by the way. It was not removed. I was just checking, like — are you guys able to resolve a simple error or not. Good."* 😄

✅ **Real lesson:** container **names must be unique**, even across stopped containers. `docker ps -a` is where you find the culprit.

◆ ◆ ◆

06 🔷 The Mess That Force-Removing Leaves Behind

⚠️ WATCH OUT

Her question after using `rm -f` : *"Do you know the consequences of doing this?"*

"The container is gone because I removed it forcefully. But my layers are still there, my content is still there — I'm just not able to see it in `docker ps` . It's still occupying my storage, still occupying my network, still occupying the volume."

The two concepts you need names for

Term	What it is
Dangling image	*"Sometimes you're using images and after a few days or months they're still there in your laptop and you're not using them — or you modified those images, or they're not needed as of now."* Untagged, orphaned image layers.
Unused container	A container that's been force-removed or whose image is gone — *"still occupying storage, network and volume"* but invisible in `docker ps` .
Zombie process	The processes **inside** an unused container: *"this container, we don't have any image, but still processes are running inside it. Still occupying the CPU and memory of my laptop. The processes, they don't have any master."*

◆ ◆ ◆

07 🧹 The Prune Commands — Reclaiming Your Disk

🧠 Her college horror story

🧠 PICTURE THIS

"In my college time I was using Docker like hell. Maybe 100s of images were there, I don't know. At that time I didn't understand Docker completely — creating images, creating containers, removing forcefully. At that time my PC was full. And then I got to know: okay, cleanup commands are there."

The commands

```
docker system prune        # ⭐ THE MASTER COMMAND
docker container prune     # only stopped/unused containers
docker image prune         # only dangling images
docker volume prune        # only unused volumes
docker network prune       # only unused networks
docker system df           # how much disk is Docker actually using?
```

What `docker system prune` asks you before it fires

```
WARNING! This will remove:
  - all stopped containers
  - all networks not used by at least one container
  - all dangling images
  - all unused build cache
Are you sure you want to continue? [y/N]
```

✅ LOCK THIS IN

Read that list before typing `y`. It's four different categories of thing, and only one of them is images.

😄 The disk-space competition

She had the whole class run it and report back:

Reclaimed	Reaction
77 KB (hers, after already cleaning that morning)	*"only 77 KB, because it's only the nginx image we ran"*
2+ GB (hers, that morning)	*"2 GB was occupied by just Docker images"*

Reclaimed	Reaction
5.7 GB	
10 GB	*"Oh my god. 10 GB. I think you are using Docker maybe on an hourly basis."* 😄
1.5 MB	*"One and a half MB?! Oh my god."* 😄

😄 CLASS MOMENT

Her benchmark, which is a genuinely funny way to measure engagement: *"Anyone more than 1 GB? If you are not getting more than 1 GB, guys, you are not learning Docker. I'm telling you."*

`docker system df` — the disk breakdown

```
docker system df
```

What her output showed:

Type	Total	Active	Size	Reclaimable
Images	27	1	**11 GB**	almost all of it
Containers	4			*(all four were Kubernetes containers)*
Local Volumes				*created for those Kubernetes containers*
Build Cache				*from a platform-ops project*

And the class's image sizes when she polled them: 20 GB · 35 GB · 32 GB · 25 GB · 4 GB · 3 GB · 2 GB.

🎤 SAID IN CLASS

"Which images are you using? I guess maybe the heavy images of Java." 😄

✅ LOCK THIS IN

The workflow she recommends: run `docker system df` **before** pruning, then again after. *"You'll be able to see you cleared 10 GB — and then I can see the huge gap."*

📸 Your own `docker system df` run

This is `Class-06/dockerdf.png` — your terminal, right after a prune:

```
WARNING! This will remove all dangling images.
Are you sure you want to continue? [y/N] y
Total reclaimed space: 0B
(base) anishkanase@Anishkas-MacBook-Air ~ % docker system df
TYPE            TOTAL     ACTIVE    SIZE      RECLAIMABLE
Images          6         4         3.027GB   40.39MB (1%)
Containers      5         5         254kB     0B (0%)
Local Volumes   0         0         0B        0B
Build Cache     16        0         637.7MB   0B
(base) anishkanase@Anishkas-MacBook-Air ~ %
```

✅ **LOCK THIS IN**

Reading your own output: 6 images totalling **3.027 GB** with only **40.39 MB (1%) reclaimable** — so your images are all actively in use, which is a healthy sign. The interesting number is **Build Cache: 16 entries, 637.7 MB** — that's over half a gigabyte of intermediate build layers from all the rebuilding you did on the four homework apps. `docker builder prune` targets exactly that.

💡 And *"Total reclaimed space: 0B"* on the prune above it means you'd already cleaned up. Good habits.

😄 Her opinion on man pages

😄 **CLASS MOMENT**

"I don't like this `man` feature, I hate it. Two pages I'm able to see — like, directly, I don't like it, literally."

Which is why she made her own: *"This is a very very specific file I created not from ChatGPT. I created it from scratch — each and every command it should cover."* (That's the `docker-basic-cmd.pdf` / `docker-advance-cmd.pdf` in the repo.)

◆ ◆ ◆

08 🐳 `docker image inspect` — And Why It Matters At Work

```
docker image inspect nginx
```

What comes back (a large JSON blob):

Field	Why you care
SHA	the image's real identity
RepoTags	*"tags are coming from that image itself, from Docker Hub"*
RepoDigests	*"these things are not necessary"*

Field	Why you care
Entrypoint / Cmd	⭐ **the important one** — see below
Networking	*"only networking stuff are necessary"* — a huge JSON section
Volumes	
OS / Architecture	*"related to the architecture of the operating system"*

🔥 The real-world story

🔥 HARD MODE

"When you're working on a live project as a DevOps engineer you need to know this entry point. Recently some developers were pinging me — the CI/CD pipeline is failing. I checked the logs and I got to know the Docker entry point is not there. So I'm telling them: add the Docker entry point, do the commit again, run the CI/CD again, and then ping me."

✅ **That's the whole job in one anecdote.** A pipeline fails → read the logs → inspect the image → the entrypoint is missing → tell the dev what to add.

What you'd see in the CMD field:

Image	CMD / Entrypoint
`nginx`	`nginx -g "daemon off;"` — *"if you add this at the end of any Dockerfile running the nginx image, it will continuously run your nginx. It will not stop until you stop it, or until something crashes."*
a Node app	`npm start` — *"it's already there, it's a default one"*

✅ LOCK THIS IN

So `docker image inspect` tells you what a container will do when you run it, without running it. That's why it's a debugging tool.

🛡️ The DevSecOps use case (this is a great story)

🎤 SAID IN CLASS

"Do you know DevSecOps? We're adding security inside DevOps. There's a specific thing we do with Docker images to secure them — at that time they use this `docker inspect` command."

What she built:

```
1. An automation tool to check vulnerabilities inside a Docker image
2. It runs  docker image inspect  and gets the JSON
3. It extracts every field into an HTML report
4. That report is what her organisation uses to review images
5. It cross-checks against a standard vulnerability DATABASE
   to verify whether the image is safe
```

✅ *"This is the use case of `docker image inspect` — if you are a DevSecOps engineer and going forward with security."*

💡 This is Class 1's **Trivy** concept, hand-built. Same idea: inspect the image, compare its contents against a known-vulnerability database.

◆ ◆ ◆

09 🐋 `docker rmi` — Removing Images, and All The Ways It Fails

🎯 "Untagged" then "Deleted" — why two lines?

```
docker rmi nginx:latest
# Untagged: nginx:latest
# Deleted: sha256:...
```

💡 GOOD TO KNOW

First, what that `sha256:...` string is. It's the image's **digest** — a fingerprint calculated from the image's actual contents. Change one byte and you get a completely different digest.

```
 nginx:latest          ← a TAG. A sticky label. Can be moved to a different image
tomorrow.
 nginx@sha256:9b1a…    ← a DIGEST. The content itself. Can never mean anything else.
```

✅ **Which is why Class 1's "you fetch it by a single SHA line" matters:** if you deploy `nginx:latest`, you can't be sure your teammate got the same bytes you did. If you deploy `nginx@sha256:9b1a…`, you both provably have the identical image. That's what "reproducible build" means in practice.

🎯 INTERVIEW ALERT

Her question: *"Why does it say untagged and then deleted? It's supposed to say only 'deleted'."*

✅ **Because a tag and an image are different things.** The tag `nginx:latest` is a *label pointing at* image layers. Docker first **removes the label**, and only then — if nothing else references those layers — deletes the actual data.

💡 Which is exactly why `docker tag nginx mynginx:v1` from Class 6 doesn't copy anything: two tags, one image.

The error when a container is using it

```
docker run -d --name mynginx1 nginx
docker rmi nginx:latest
```

```
Error response from the daemon: conflict: unable to delete nginx:latest
(must be forced) - image is being used by running container ...
```

🎯 INTERVIEW ALERT

She dwelt on the wording: *"Error response from the daemon. So this is what the daemon means — the daemon is handling everything."*

Your two options:

```
docker stop mynginx1 && docker rmi nginx:latest   # ✅ the polite way
docker rmi -f nginx:latest                        # ⚡ the forced way
```

💀 And what forcing it leaves you with

She forced it. Then:

```
docker ps -a
```

⚠️ WATCH OUT

"Container is still running — but I can see there is no name of the container, there is no image for the container. That is what we call an unused container. And inside this unused container, there are zombie processes — this container doesn't have any image, but processes are still running inside it, still occupying the CPU and memory of my laptop."

And removing it takes two steps, which caught her out live:

```
docker container prune    # ❌ didn't remove it
docker ps -a              # still there!
docker stop <id>          # ← because it was still RUNNING
docker container prune    # ✅ now it's gone
```

✅ LOCK THIS IN

`prune` **only removes *stopped* containers.** A running one — even a zombie with no image — survives it.

🎤 *"This is the learning, guys. If you are understanding, if you are doing — then you can understand what exactly is going on."*

🎯 Why she spent 20 minutes on error messages

🎯 INTERVIEW ALERT

Her explanation is the most useful careers advice in the class:

"These types of questions they ask in interviews. They will not give you the time to test whether it will run or not. They'll just ask: 'if I directly remove the container without stopping it, what will happen?' At that time you need to tell them."

The real interview she had:

```
Interviewer: "A container is running and I want to delete the image. What happens?"
Her:         "You'll get an error."
Interviewer: "What error?"
Her:         (described the conflict error)
Interviewer: "From which COMPONENT is the error coming?"
Her:         "From the daemon."
Interviewer: "Okay — nothing more I can ask you."   ✅
```

✅ *"So you need to know which kind of error you're getting and from which component. And to understand it you need to run the command, get the error, and resolve the error. This is the only way we can learn Docker — even for Kubernetes, Terraform, everything in DevOps."*

◆ ◆ ◆

10 🐳 `docker exec` — Getting Inside a Running Container

🎯 INTERVIEW ALERT

The scenario: *"The container is running already and I want to edit, update or delete — any operation I want to do inside a container which is already running. I don't want to stop it."*

🧠 **The analogy:** *"If your application is running on a server, first you log into the server, then you check your code, then you check the logs. That's how you're doing it."* `docker exec` is SSH-ing into a container.

The command

```
docker exec -it <container-id-or-name> /bin/bash
#           |                          └── which shell to open
#           └── ALWAYS -it   ("always, always for -it")
```

🧠 It's a tiny Linux box in there

Once inside, you get a root prompt and **every Linux command from Class 2 works**:

```
pwd     # →  /
ls      # →  bin boot dev etc home lib media mnt opt proc root
        #    run sbin srv sys tmp usr var
cd root
```

✅ LOCK THIS IN

"Inside the container we have a Linux system. See, the forward slash is there. It's like a mini Linux instance that we just logged into."

🧠 And she pointed at the payoff: *"*`bin`, `boot`, `media`, `home`, `root`, `sys`, `tmp`, `usr`, `var`, `mnt`, `lib` *— everything that we have in my laptop, the same thing I can see here."* Class 2's filesystem tour, inside a container.

🎯 The interview question hiding in the prompt

Look at the prompt after you exec in:

```
root@9594518fce35:/#
     └──────────┘
       this is the CONTAINER ID
```

🎯 INTERVIEW ALERT

The question, which she said came from a LinkedIn post about a real company's interview: *"If you want to check the hostname of a container without using* `exec`*, how can you check?"*

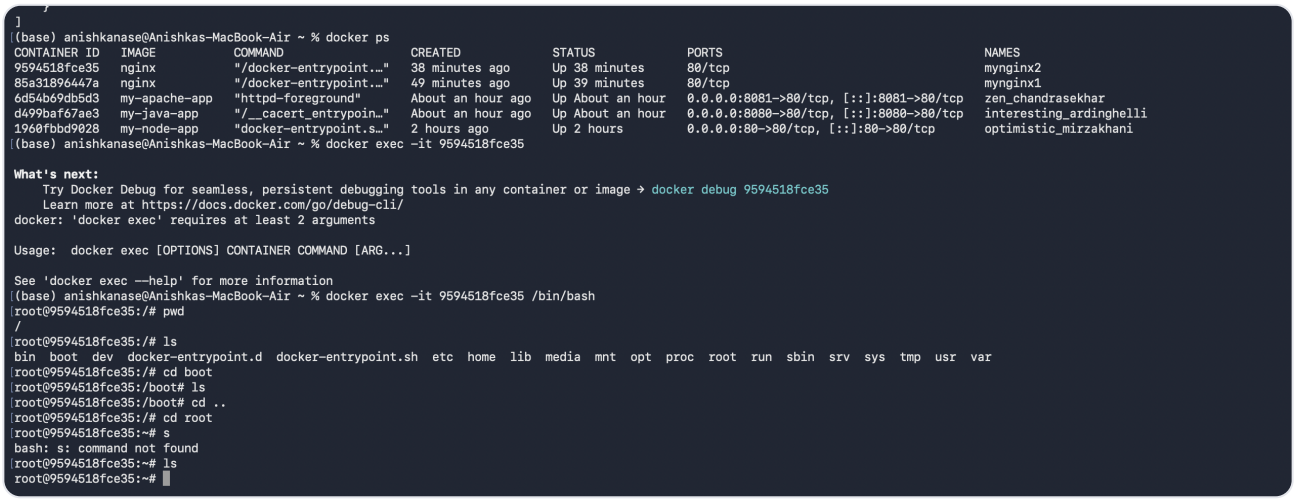
✅ **Answer: the container's hostname IS its container ID.** They're the same string. So `docker ps` already tells you the hostname.

🎤 *"I was just curious, like — without doing the exec command, how can we check? Then I got to know there's a concept: the hostname and the container ID both are the same."*

📌 Also note: `root` in that prompt is the **username**, and the hex string is the **hostname**. The class initially answered "hostname" for `root` — it's the other way round.

📸 Your actual `docker exec` session

This is `Class-06/exec.png` — and it's a great artifact, because it shows all four homework apps running at once:



What's in that screenshot, decoded:

Container	Image	Ports	Name
`9594518fce35`	nginx	`80/tcp`	**myng inx2**
`85a31896447a`	nginx	`80/tcp`	**mynginx1**
`6d54b69db5d3`	**my-apache-app**	`8081→80`	`zen_chandrasekhar`
`d499baf67ae3`	**my-java-app**	`8080→80`	`interesting_ardinghelli`
`1960fbbd9028`	**my-node-app**	`80→80`	`optimistic_mirzakhani`

✅ **LOCK THIS IN**

Three things your screenshot proves nicely:

1. **You named the nginx ones (** `mynginx1` **,** `mynginx2` **) but not the app ones** — hence `zen_chandrasekhar` and `interesting_ardinghelli`. That's the random-name generator from §5, live.
2. **The first** `docker exec -it 9594518fce35` **failed** with `docker: 'docker exec' requires at least 2 arguments`. Correct — you must say *what to run* inside. The next line, `docker exec -it 9594518fce35 /bin/bash`, worked.
3. **Then** `pwd` **→** `/` **,** `ls` **→ the full Linux tree,** `cd boot` **,** `cd ..` **,** `cd root` — exactly the exploration she asked for. (And a stray `s` → `bash: s: command not found`, which happens to everyone. 🙂)

💡 **One detail worth noticing:** the two nginx containers show `80/tcp` with **no** `0.0.0.0:` **prefix** — meaning they were started **without** `-p`, so nothing is published to your laptop. The three app containers show `0.0.0.0:8081->80/tcp` etc., which is what a real port mapping looks like.

📝 The task

📝 **Run an nginx container, `docker exec` into it, run `ls`, `pwd` and `cd root`, screenshot it, put it in the same README, and commit.** She wanted 5–6 repos, specifically from Team A and Team C.

◆ ◆ ◆

11 🔷 Class Engagement (The Comedy Section)

Worth keeping — it's a lot of the class's character.

😄 **CLASS MOMENT**

On file naming: a student pushed a file called `pic.md`. *"MD file, PNG — right? Or MD file? Don't create a file something like `pic.md`. This is not a good approach. Huh? `pic.md`? Okay. Nice, nice, good."* 😄

😄 **CLASS MOMENT**

On the student named "improvement": *"Improvement — I like that name. Means improvement is always an improvement."*

😄 **CLASS MOMENT**

On someone linking a Google Drive instead of committing: *"I mean, literally crazy — you added a link of a Drive, and inside that also a `docker system df` command which we already ran 10 to 15 minutes back. Can you add an MD file instead of directly a Drive?"*

😄 **CLASS MOMENT**

On the reluctant teams: *"Team C, guys, look at me… I need your repo… He's watching or not? …This team B is waiting, you are wasting our time. Come on, Team A, Team C… like this I will find who is not doing."*

And the closer: *"We can change the world, we can change ourselves — but the program team can change you. Surely, that I know. So I'll tell them."* 😄

💡 The GitHub `blame` tangent (genuinely useful)

> 💡 **GOOD TO KNOW**
>
> *"I like this feature of GitHub — I can check who added the code changes."*
>
> **And its cousin in Azure DevOps:** *"In ADO — Azure DevOps portal — if you do a Git commit there, there's one tab over any file. If I change something in this file, I can check the blame."*
>
> 😄 *"I literally use it to blame someone. Something they changed and my CI/CD is not working — I'll just blame. See, this guy, this profile — you can copy the profile, post it in chat, talk with him. He added something and then your pipeline is not working."*
>
> ✅ **The real skill here:** `git blame` (or the Blame tab on GitHub/ADO) tells you *which commit* last touched a line and *who* wrote it. When a pipeline breaks after a merge, that's your fastest route to the cause.

◆ ◆ ◆

12 🔁 Dockerfile Instructions — Recap With New Details

She re-walked all the instructions, and three of them got genuinely new material.

🆕 `ENV` vs `ARG` — the real difference

	`ENV`	`ARG`
Available at	**run** time (and build time)	**build** time only
Set inside the Dockerfile	`ENV NODE_ENV=development`	`ARG NODE_ENV=development`
Overridden from the CLI	via `docker run -e`	via `docker build --build-arg`

```
docker build --build-arg NODE_ENV=production -t myimage .
#            └──────────────────────────────┘
#               this is ARG's whole reason to exist
```

> ✅ **LOCK THIS IN**
>
> **Her framing:** *"If you want to give an environment-specific variable, use `ENV` inside the Dockerfile. But if you want to apply the argument — then when we're doing `docker build -t`, at that time I'll add `--build-arg` and the same argument name."*

🆕 `USER` — the security instruction nobody remembers

She exec'd into a container and pointed at the prompt:

```
root@9594518fce35:/#
└──┘
  the DEFAULT user is root
```

```
FROM node:24-alpine
...
USER node          # ← add this
CMD ["npm", "start"]
```

> ⚠️ **WATCH OUT**
>
> **Why it matters:** *"If we did not specify any user inside the Dockerfile — this is a Node.js application, so we enter `node`. Whenever someone execs inside my container they are not able to see root, because root user has all these sudo privileges."*
>
> ✅ *"So you should never, ever directly run like this. Instead, you need to add `USER` inside your Dockerfile for security purposes."*
>
> 💡 **This is "shift left" from Class 1, in one line of a Dockerfile.** A container running as root means a container escape gets root. Adding `USER` costs nothing.

🆕 A new detail on `CMD` vs `ENTRYPOINT`

The familiar rule:

	Overridable?
`CMD`	✅ yes
`ENTRYPOINT`	❌ no

And the new bit she added:

> 📌 **NOTE**
>
> *"When you stop a container and start it, it will execute the ENTRYPOINT. If you restart a container, it will execute the CMD."*
>
> 🔍 **Worth a note, because this is easy to over-apply:** in practice Docker re-runs the container's *full* configured command (`ENTRYPOINT` + `CMD` together) on both `start` and `restart` — the two aren't split across the two commands. Her point is best held as the thing she said elsewhere and is definitely true: `ENTRYPOINT` **always runs, no matter how the container comes up;** `CMD` **is the part a** `docker run` **argument can replace.** For her assessment, give the overridable/not-overridable answer — that's the one being tested.

`COPY` vs `ADD`, once more

	`COPY`	`ADD`
Local files	✅	✅
URLs / remote files	❌	✅
Auto-extract zip / tar / RPM	❌	✅

The full instruction table

Instruction	Does
`FROM`	base image + tag
`WORKDIR`	*"set the working directory inside an image"*
`COPY src dst`	local → container
`ADD src dst`	COPY + download + auto-extract
`RUN`	run a command at **build** time
`ENV k=v`	environment variable (run time)
`ARG k=v`	build-time argument (`--build-arg`)
`EXPOSE port`	declare the port
`USER`	🆕 which user the container runs as — **not root**
`CMD [...]`	start command — overridable
`ENTRYPOINT [...]`	start command — not overridable

◆ ◆ ◆

13 📁 Multi-Stage Dockerfiles — Today's Main Topic

🎯 Why they exist — and a student nailed it instantly

🎯 **INTERVIEW ALERT**

Her setup: *"There was a problem: if you're creating a container for a Node.js application, a Java application — one thing is related to storage, which you all faced. Right? You all saw 30 GB, 50 GB, 10 GB."*

Student, immediately: *" `node_modules` ."*

🎤 *"Exactly. He understood why we need a multi-stage Dockerfile."*

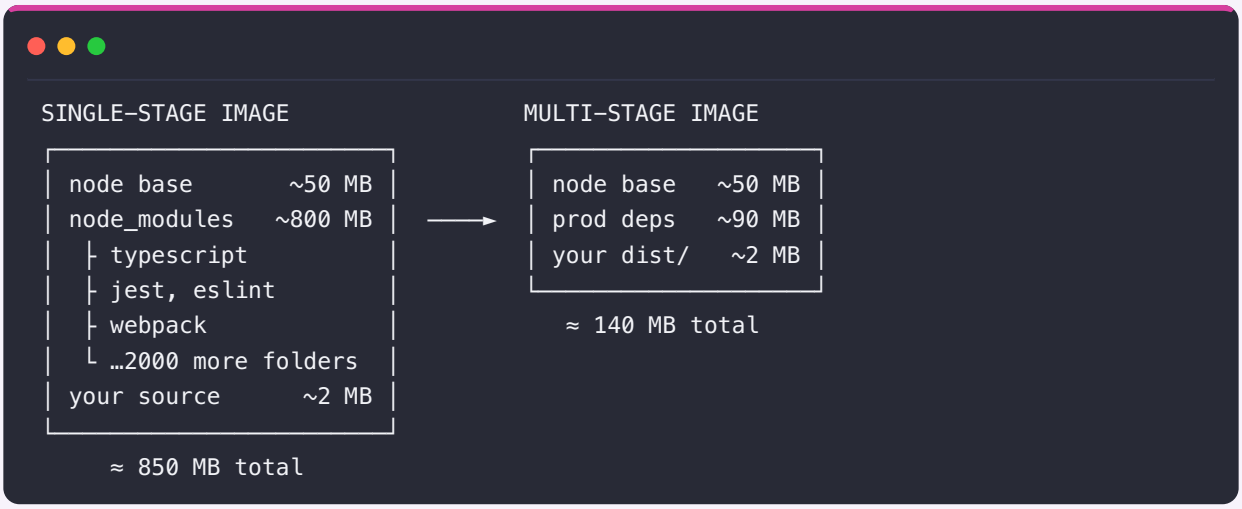
The problem, precisely

```
npm install
   └─► node_modules/     ←  1 GB? 2 GB? 3 GB?
   └─► build/  or  dist/  or  target/   ←  the ONLY bit production needs
   └─► test/              ←  also not needed in production
```

🧠 PICTURE THIS

Why `node_modules` is so absurdly large — and what you're actually throwing away.

Your app might depend on 10 packages. But each of those depends on 20 more, and each of *those*… so you end up with thousands of folders. Worse, most of it is **build-time only**: compilers, test frameworks, type definitions, linters. None of it runs in production.



✅ **Three wins from one technique:** the image is ~6× smaller (faster to push, pull and start), your **source code never ships** (only the built output does), and **Trivy has far less to scan** — every dev dependency you drop is one less possible vulnerability.

✅ LOCK THIS IN

"We don't want to take all the files and storage — maybe 1, 2, 3 GB. We can't take that in production. Those files are not necessary. We just need the build folder."

💡 **Folder names by ecosystem:** Node → `build/` or `dist/` · Java/Maven → `target/` . *"And sometimes if you're adding test cases then the test folder is already there — that's what I've seen in my organisation."*

The Dockerfile

```dockerfile
# -------------------------
# Stage 1: Build
# -------------------------
FROM node:24-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .

# -------------------------
# Stage 2: Production
# -------------------------
FROM node:24-alpine AS production
WORKDIR /app
COPY --from=builder /app/package*.json ./
RUN npm install --omit=dev
COPY --from=builder /app/server.js ./
EXPOSE 3000
CMD ["npm", "start"]
```

✅ Only TWO things make it multi-stage

✅ LOCK THIS IN

She repeated this four times, because it really is the whole trick:

```
1.  AS builder            ← on the first FROM, names that stage
2.  COPY --from=builder   ← in the second stage, pulls files OUT of stage 1
```

"For multi-stage you just need one thing: `AS builder`. Second thing, in the runtime stage: `--from=builder`. Two things only to create a multi-stage Dockerfile."

Reading it stage by stage

Stage 1 — builder:

Line	What it does
`FROM node:24-alpine AS builder`	base image, **named** `builder`
`WORKDIR /app`	work inside `/app`
`COPY package*.json ./`	bring in the dependency manifest
`RUN npm install`	creates the giant `node_modules` inside `/app`

Line	What it does
`COPY . .`	bring in the source
(and in a real app) `RUN npm run build`	produces `dist/`

Stage 2 — production: a **completely fresh image**. Nothing from stage 1 exists here unless you explicitly copy it.

Line	What it does
`FROM node:24-alpine AS production`	fresh, empty base
`COPY --from=builder /app/package*.json ./`	⭐ reach back into stage 1
`RUN npm install --omit=dev`	install **production deps only**
`COPY --from=builder /app/server.js ./`	⭐ copy **only** the built output
`EXPOSE 3000`	
`CMD ["npm", "start"]`	

✅ **LOCK THIS IN**

The result: `node_modules` from the build (with all the dev tooling, compilers, test frameworks) **never enters the final image.** You ship the output, not the workshop.

💡 Two ways to say "production only"

```
RUN npm install --omit=dev      # (older syntax: --only=production)
```

or

```
ENV NODE_ENV=production
RUN npm install
```

✅ **LOCK THIS IN**

"Previously we added `ENV NODE_ENV=development`. If you want to make it production, you write `--only=production` here. If you don't want to write it, you can add `ENV NODE_ENV=production` inside your Dockerfile. If you're not adding it inside your Dockerfile, then you need to add `--only=production`." Either works — that's the `ENV` from §12 paying off.

⚠️ Multi-stage ≠ multi-container

⚠️ WATCH OUT

She flagged this twice across two classes: *"Multi-stage is a different thing, multi-container is a different thing. Two different topics we need to learn."*

	Means
Multi-stage	**one** Dockerfile, **one** final image, multiple `FROM` blocks — a *build technique*
Multi-container	**many** containers running together — that's **Docker Compose**, Class 9

💡 GOOD TO KNOW

Also possible: the two stages don't have to use the same base image. *"You can use a different image. There are no constraints — you can use anything else. Just two things: `AS builder` and `--from`."* A classic real-world combo is build with `node`, serve with `nginx`.

🧪 The live build (do this yourself)

```
git clone https://github.com/Nency-Ravaliya/devops-heros.git
cd devops-heros/session6-7-docker/multi-stage-dockerfile
ls                                          # Dockerfile, package.json, server.js

docker build -t multi-stage-node-app .      # build
docker images                               # → multi-stage-node-app  latest

docker run -it -d -p 8080:3000 multi-stage-node-app:latest
```

🎯 INTERVIEW ALERT

Her port quiz, again: *"`8080` — is it the container port or the host port?"* → **host.** *"And `3000` ?"* → **container** (because `EXPOSE 3000`).

Then open `localhost:8080` → *"Hello World from Docker Multi-Stage Build!"* 🎉

◆ ◆ ◆

14 📝 Homework

#	Task
1	🏗️ **Run the multi-stage Dockerfile yourself.** Clone her repo, build it, run it.

#	Task
2	📸 **Two screenshots** in an `.md` file: (a) the app in the **browser**, (b) `docker ps` showing it running on port **8080**
3	🧹 Screenshot your `docker system df` output
4	🐚 Screenshot your `docker exec` session (`ls` , `pwd` , `cd root`)
5	🧪 Test `start` **vs** `restart` yourself after a `docker stop`
6	🧪 Test `docker rm` on a running container — get the error, then fix it
7	☕ Finish the **four apps** from Class 6 if you haven't — especially **Java**

💡 GOOD TO KNOW

Her minimum bar: *"Bare minimum, at least you need to run your application. That's it."* Plus your name, enrolment number and a comment in the file.

🎓 The Java pep-talk

She polled who'd finished the four apps — answers ranged from *"three of three"* to *"five"* to silence.

😄 CLASS MOMENT

"So I assume you either did three, four, five — or you did not do anything. Again I'm asking: at least three applications, how many of you did? …What about others? Okay, this is the second concern I have to tell them." 😄

✅ LOCK THIS IN

But then a genuinely motivating point about Java:

"Only Java application, I guess, is remaining from your side. Who did it? Congratulations — because one of the tough applications is Java. Because we have legacy code for Java in multiple organisations. In Bangalore we have a lot of organisations that have old Java code and they want to containerise it. Those kinds of DevOps engineers, they want. And if you did the Java application, that means you have a certain set of understanding of how to containerise a Java application."

💡 That's the **legacy application** use case from Class 6, and it's a real, paid, in-demand skill — not a textbook example.

◆ ◆ ◆

15 🔁 Her Own Recap

🎤 SAID IN CLASS

"What we covered today so far: Docker images, containers, commands, prune commands, container lifecycle. We had a complete hands-on of how a container lifecycle works — from created stage to start, to pause, to run, to delete. Then we deep-dived: how to stop, how to remove, forcefully remove images and containers. We cannot delete images directly unless we do it forcefully — those troubleshooting concepts we did. Then how to `exec` inside a container. Then the last thing, multi-stage Dockerfile, and how to run it."

◆ ◆ ◆

16 📔 Cheat-Sheet

Lifecycle

Command	Does
`docker build -t <name> .`	Dockerfile → image
`docker container create --name <n> <img>`	image → **created** (not started)
`docker start <n>`	created/stopped → **running**, from PID 1
`docker run -d --name <n> -p H:C <img>`	create **+** start, in one command
`docker pause <n>` / `docker unpause <n>`	freeze / unfreeze internal services
`docker stop <n>`	graceful shutdown → **exited**
`docker restart <n>`	resume from the stopped state ⭐
`docker rm <n>`	remove (must be stopped first)
`docker rm -f <n>`	force-remove a **running** container
`docker ps`	**running** only
`docker ps -a`	**all**, including exited ⭐
`docker logs <n>`	see output from a detached container

Images

Command	Does
`docker pull <img>`	download an image

Command	Does
`docker images`	list local images
`docker image inspect <img>`	full JSON — entrypoint, CMD, networking, volumes
`docker tag <img> <new:tag>`	add another tag (no copy happens)
`docker rmi <img>`	untag **then** delete
`docker rmi -f <img>`	force, even if a container is using it

Inside a container

Command	Does
`docker exec -it <n> /bin/bash`	open a shell inside ⭐
`docker exec -it <n> <command>`	run one command inside
(inside) `pwd` `ls` `cd` `cat`	it's a mini Linux

Cleanup

Command	Removes
`docker system prune`	⭐ stopped containers + unused networks + dangling images + build cache
`docker container prune`	**stopped** containers only
`docker image prune`	dangling images only
`docker volume prune`	unused volumes
`docker network prune`	unused networks
`docker builder prune`	build cache only
`docker system df`	how much disk Docker is using (run it **before** pruning)

Multi-stage

Keyword	Does
`FROM <img> AS builder`	name a stage
`COPY --from=builder <src> <dst>`	pull files out of a named stage
`RUN npm install --omit=dev`	production dependencies only
`ENV NODE_ENV=production`	the alternative to the flag above

Concepts

Term	Means
Immutable	image can't change — edit code → **rebuild**
PID 1	the container's main process; kill it → container dies
Created state	container exists, PID 1 not started
Paused	services frozen, container still running
Exited	stopped gracefully, state saved
Dangling image	untagged, orphaned image layers
Unused container	a container with no image — still eating storage
Zombie process	a process inside an unused container, with no master
`--name`	name your container, or get `competent_thompson`
Detach mode (`-d`)	run in the background — *"always, always"* on servers
Container hostname	🎯 **the same string as the container ID**
Docker host	any machine running Docker — including every Kubernetes worker node
`docker blame`	doesn't exist 🙂 — but `git blame` does, and it's how you find who broke the pipeline

◆ ◆ ◆

17 🗒️ Notes on the Transcript Itself

- **Instructor name:** the header credits *"Ritesh Prajapati"* — the **Scaler++ Chrome extension's developer**, not the teacher. The instructor is **Nensi Ravaliya** (@Nency-Ravaliya).
- **This recording is much cleaner than Classes 1–6**, because the transcription switched to **Whisper large-v3-turbo** (visible in the header). Almost no foreign-language artifacts.
- Garbled terms corrected: **"demon" → daemon**, **"docker exit command" →** `docker exec`, **"docker prone / prove" →** `docker prune`, **"docker system def" →** `docker system df`, **"talker" → Docker**, **"RM state" → removed state**, **"post it / post step" → paused**, **"nerd modules" →** `node_modules`, **"plastic ops" → a platform-ops project**, **"ADO" → Azure DevOps**, **"Booker" → Docker (her typo, live)**, **"AST, AST" → the random container-name generator**, **"multi build / multi Dockerfile" → multi-stage build**, **"buildup" →** `builder` **stage**, **"as buildup" →** `AS builder`.
- 😄 **Two asides kept because they're funny and human:** her typing speed — *"I'm a hard and fast typer… I type on my keyboard like someone is just putting their complete frustration of a whole month"* — and the giant-enter-button reel her friends sent her. She also apologised for typos roughly six times.